COMPLAINT ANALYZER SYSTEM

Complete System, AI, Roles, Setup and Operations Guide

A privacy-safe explanation of how the platform works from first visit to final resolution.

Prepared from the current application source and configuration
Version 1.0 | 24 August 2026

How to Use This Guide

This document explains the purpose, architecture, roles, AI behavior, complaint workflow, database, APIs, installation, deployment, security, and daily operation of the Complaint Analyzer System. It is suitable for project reviewers, university stakeholders, operators, developers, and authorized support personnel.

Privacy boundary
This guide intentionally contains no real account names, email addresses, registration numbers, passwords, tokens, complaint text, attachments, IP addresses, device records, or database rows. Examples describe structure and behavior only.

Key takeaway

The platform is a role-based university complaint workflow. Students submit and track issues, authorized staff process them, administrators monitor the system, MariaDB stores the official record, and AI provides advisory classification and guidance. Human reviewers remain responsible for decisions.

Document conventions

- Student refers to the person who registers with a university registration number and a personal Gmail address.
- Admin area refers to the separate login and dashboards used by administrative and departmental roles.
- Live data means the page requests current API and MariaDB results instead of showing fixed demonstration numbers.
- AI result means an automated estimate. It is not a final decision, proof, or disciplinary judgment.
- Production means a real university deployment with HTTPS, protected evidence storage, managed secrets, backups, and approved privacy rules.

Contents

1. System at a Glance

The Complaint Analyzer System provides one connected workflow for public visitors, students, administrators, and department members.

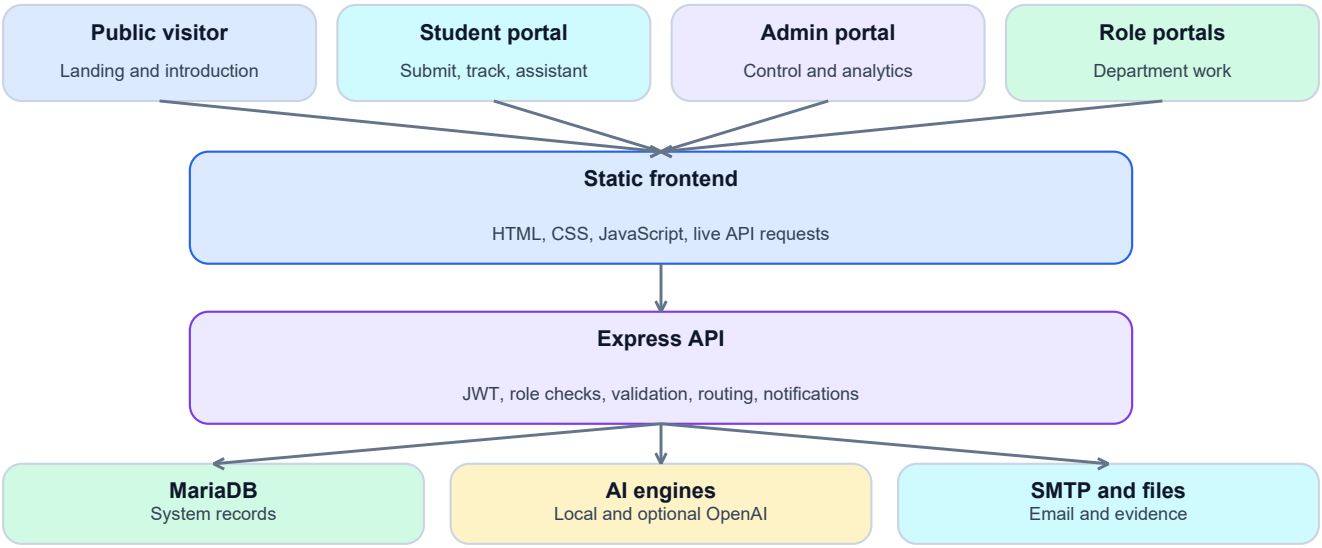


Figure 1. High-level system architecture.

What the system solves

- Gives students a structured channel for reporting academic, fee, administrative, and other concerns.
- Keeps complaint status, notes, evidence, notifications, and history in one record.
- Routes issues to an appropriate department using rule-based analysis and optional external AI.
- Provides live counts, recent complaints, analytics, predictions, exports, and activity tracking for authorized roles.
- Separates student access from administrative access and applies backend role checks to protected operations.
- Records audit and request metadata to support accountable review without treating metadata as proof.

Main application areas

Area	Purpose	Typical functions
Public site	Introduces the service before sign-in.	First-visit overview, workflow explanation, public live summary.
Student portal	Creates and follows student complaints.	Registration, login, submission, evidence, history, status, notifications, assistant.
Admin portal	Controls and monitors the complete complaint process.	Complaint review, status actions, analytics, roles, reports, predictions, audit and activity.
Department portals	Supports scoped operational work.	Assigned work, status changes, notes, and department activity.
Backend API	Enforces business rules and access.	Authentication, validation, analysis, database operations, files, email, and responses.

2. Architecture and Technology

Frontend

The browser interface uses static HTML, CSS, and JavaScript. Express serves these files from the same application. Browser scripts call JSON endpoints under the `/api` path, render live database values, maintain session state, and route each role to its dashboard.

Backend

Node.js and Express 5 provide the web server and API. The backend is organized into routes, controllers, models, middleware, AI services, and utilities. Routes identify the endpoint, middleware verifies identity and permissions, controllers coordinate the use case, and models execute parameterized SQL.

Database and storage

MariaDB is the verified local database engine. The `mysql2` driver provides a pooled connection. Complaint evidence is stored on disk under the upload directory, while attachment metadata and file paths are stored in the database. Production evidence storage should be private and persistent.

Main technology inventory

Technology	Role in the system
Node.js	Application runtime for the API and static file server.
Express 5	Routing, middleware pipeline, JSON handling, and HTTP responses.
MariaDB / MySQL protocol	Persistent structured records and analytics queries.
JWT	Signed bearer tokens used by protected API routes.
bcryptjs	Password hashing and password comparison.
Helmet	Browser security headers, including a Content Security Policy.
express-rate-limit	Limits repeated authentication and verification requests.
Multer	Validates and stores evidence uploads.
Nodemailer	Verification, welcome, and password-reset email delivery.
Local NLP rules	Always-available complaint classification and assistant fallback.
Optional OpenAI API	Enhanced classification and assistant replies when explicitly enabled.

Current frontend deployment note

The browser JavaScript currently points to the local API address. Before remote deployment or access from another device, configure it to use the production origin or a relative `/api` base path.

3. First Visit and Student Journey

First visit

A first-time visitor is redirected from the landing page to a welcome introduction. The browser records that the introduction has been seen, so normal future visits do not repeatedly show it. A returning student with a stored valid session is redirected to the student dashboard.

Registration

1. The student opens the separate student registration page.
2. The form collects a full name, personal Gmail address, university registration number, and password.
3. The registration number is normalized to uppercase with spaces removed.
4. The server checks Gmail format, registration-number format, required fields, password length, and uniqueness.
5. When verification is enabled, a six-digit code is sent through SMTP and must be supplied before expiry.
6. The password is hashed before the new account is written to MariaDB.
7. The server returns a student token and attempts to send a welcome email when SMTP is configured.

Login and return visits

A student can sign in using either the registered Gmail address or the normalized university registration number. Student session values are stored in browser local storage so a returning student can move directly to the dashboard until logout, token expiry, or browser storage removal.

Student dashboard

- Total, pending, resolved, and high-priority complaint counts come from the student's MariaDB records.
- Recent complaints and the complaint summary are loaded through the API.
- Navigation opens submission, all complaints, status, notifications, and assistant pages.
- No fixed demonstration complaint should be treated as live data.

4. Authentication and Account Rules

Identity controls

Control	Current behavior	Purpose
Unique Gmail	One student account per Gmail address.	Prevents duplicate student identities using the same mailbox.
Unique registration number	One account per normalized university registration number.	Links registration to one student account.
Verification code	Six digits, hashed in storage, expires after ten minutes, limited attempts.	Confirms mailbox control when enabled.
Password hashing	bcrypt hash is stored instead of the original password.	Protects the password database against plain-text disclosure.
JWT	Protected calls send a bearer token.	Carries account identifier and role for backend authorization.
Password reset	Time-limited reset token sent by email.	Allows recovery without revealing whether an arbitrary account exists.

Separate session behavior

- Student sessions use local storage for convenient return visits.
- Admin and staff sessions use session storage and therefore do not intentionally persist as long as student sessions.
- Logout removes both student and administrative session keys.
- The backend verifies every protected token and does not rely only on hidden buttons or page redirects.

Rate limits

The authentication API has a general request limit. Verification-code requests have a stricter limiter and a resend delay. These controls reduce automated abuse and explain why a user may see a 'too many requests' response after repeated attempts. The correct action is to wait for the stated period rather than continuously retry.

Do not disclose account information
Support staff should never place passwords, tokens, verification codes, full account lists, or private student details in public documentation, screenshots, tickets, or chat messages.

5. Complaint Lifecycle

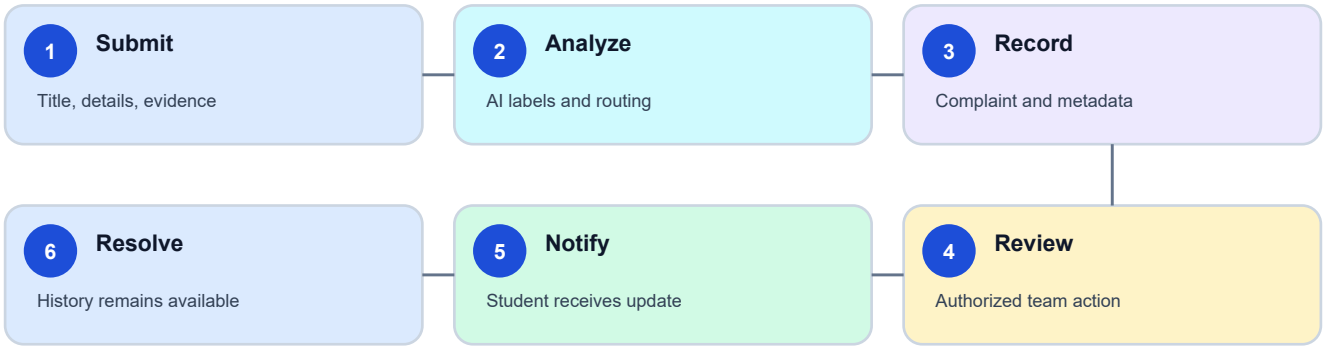


Figure 2. Complaint processing from submission to resolution.

Submission

A complaint requires a title and description. A manual category is optional. Evidence is optional. The backend analyzes the combined text, chooses or respects a category, recommends a department, stores the initial Pending status, writes tracking and notification records, logs an audit event, and records available request metadata.

Status states

Status	Meaning	Expected action
Pending	The complaint has been received but work has not been recorded.	Review category, priority, evidence, and ownership.
In Progress	An authorized person or department is actively processing the complaint.	Add a clear note and continue follow-up.
Resolved	The complaint has reached a recorded completion state.	Keep the history and communicate the outcome.

Follow-up behavior

- Every status change can include an explanatory note.
- The previous status, new status, actor, note, and timestamp are recorded in status history.
- The complaint owner receives a notification after an authorized status action.
- Department members can add work notes when the complaint is within their authorized scope.
- A potential-false flag requires a reason and notifies the complaint owner; it remains a review state, not an automatic verdict.

6. Roles and Responsibilities

Role-based access separates who can submit, process, review, administer, and monitor complaints. The effective permission comes from backend middleware and department scope, not from the visual dashboard alone.

Role definitions

Role	Primary responsibility	Main permissions	Important limit
Student	Create and follow personal complaints.	Register, sign in, submit, attach evidence, view own history and notifications, use assistant.	Cannot access another student's complaint or administrative controls.
Super Admin / Main Admin	Operate the complete system.	All-complaint review, analytics, predictions, audit, role management, exports, account and complaint deletion.	Must use least privilege and documented governance.
Admin	Central complaint administration.	Review complaints, update status, view dashboards and allowed management functions.	Actions remain subject to backend role rules.
Department Admin	Manage department-assigned work.	View authorized department complaints, update work status, add notes.	Cannot operate outside assigned scope or manage the full system.
Faculty Staff	Process authorized academic or departmental tasks.	View scoped work and add operational notes or updates.	No global account or analytics authority.
Reviewer	Review assigned complaint work.	Inspect scoped complaints and contribute notes or review actions.	No global administrative control.

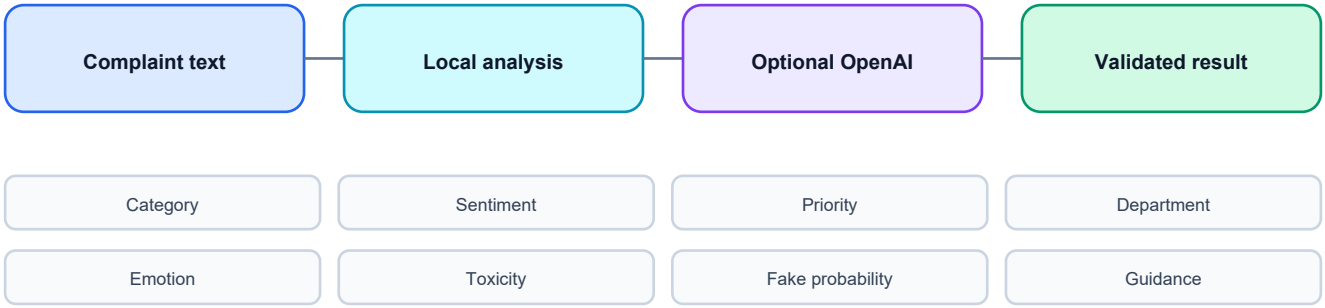
Permission matrix

Capability	Student	Main Admin	Dept. Admin	Faculty	Reviewer
Submit complaint	Yes	No	No	No	No
View own complaint	Yes	All	Scoped	Scoped	Scoped
Update complaint status	No	Yes	Scoped	Scoped	Scoped
Add work note	No	Yes	Scoped	Scoped	Scoped
Manage role accounts	No	Yes	No	No	No
View full analytics	No	Yes	No	No	No
Export all complaints	No	Yes	No	No	No
Use student assistant	Yes	No	No	No	No

Governance rule

A privileged role should receive only the access required for its job. Account creation, complaint deletion, exports, audit access, and false-complaint decisions should be limited, reviewed, and logged.

7. The Role of AI



Fallback keeps analysis available; human review owns the decision.

Figure 3. AI analysis with an always-available local path and optional external enhancement.

What AI does

Output	Meaning	How it helps
Category	Closest issue type such as fee, academic, administration, or other.	Supports queue organization and department routing.
Sentiment	Positive, neutral, or negative tone estimate.	Adds context for reporting and review.
Priority	High, medium, or low urgency estimate.	Helps teams review urgent-looking issues earlier.
Department	Recommended operational destination.	Reduces manual sorting.
Confidence	A numeric estimate of classification certainty.	Shows that automated output is probabilistic.
Emotion	A broad emotional indicator.	Provides supporting context for a reviewer.
Toxicity score	Keyword-based estimate of hostile language.	Can prompt careful moderation without deciding guilt.
Fake probability	Phrase-based uncertainty signal.	Requests human verification; never proves falsity.
Expected time and recommendation	Suggested handling window and next action.	Provides operational guidance, not a guaranteed service level.

Local analysis

The local engines use transparent keyword scoring. They count words associated with categories, positive or negative sentiment, urgency, hostility, and uncertain claims. They then assign labels and a confidence estimate. This method is fast, works offline, and provides a fallback when no external service is configured. Its limitation is that keywords cannot fully understand context, sarcasm, language variation, or complex evidence.

Optional OpenAI analysis

When external AI is enabled and a valid API key is present, complaint text can be sent to the configured OpenAI model for category, sentiment, priority, and confidence output. The backend requests JSON and falls back to local rules if the call fails or the result cannot be used. Enabling this option means complaint text leaves the university server, so legal approval, a data-processing review, and a clear privacy notice are required before production use.

Complaint assistant

- Answers questions about the student's complaint status and general next steps.
- Uses a small summary of recent complaint records and recent conversation messages as context.
- Provides local, friendly responses when external AI is disabled or unavailable.
- Is instructed not to invent a status, promise an outcome, or request a password.

- Guides urgent safety issues toward immediate university support when appropriate.

AI accountability
AI must not be the sole basis for rejection, discipline, a false-complaint finding, or final resolution. A trained human reviewer must examine the complaint, evidence, history, and applicable policy.

8. Categories and Department Routing

Detected category	Typical indicators	Recommended destination
Fee Issue	Fee, payment, challan, dues, voucher, refund, installment, scholarship.	Finance Office
Academic Issue	Class, teacher, exam, marks, course, attendance, assignment, result, lab.	Academic Department
Administration Issue	Office, staff, document, certificate, transcript, admission, registration.	Administration Office
Other	No strong match to the configured categories.	Student Affairs

Routing process

1. The analyzer scores category keywords in the title and description.
2. The highest matching category becomes the detected category unless an allowed manual category is supplied.
3. The category maps to a recommended department.
4. The complaint stores its department identifier when a matching department record exists.
5. Department-role middleware limits operational views to authorized work.
6. A human can review whether the route is appropriate before final handling.

Routing is assistance, not ownership policy
The recommended department is an operational suggestion. University policy should define who can transfer, accept, escalate, or close a complaint.

9. Dashboards, Analytics and Live Data

Student live data

- Summary cards count only complaints owned by the logged-in student.
- All Complaints loads the complete student history from MariaDB.
- Recent Complaints is a current subset ordered by creation time.
- Complaint Summary describes the student's live complaint distribution and state.
- Status and notifications reflect authorized follow-up actions.

Administrative live data

- Dashboard totals, today's volume, high priority, and resolved counts are database queries.
- Complaint management supports live search and filters by category, priority, and status.
- Analytics aggregates category, sentiment, priority, and monthly trends.
- Advanced analytics and predictions summarize current records rather than fixed example values.
- Member activity reports work actions and notes for accountability.
- CSV export provides an authorized report for offline analysis and record handling.

How to interpret empty values

A zero can be correct when no matching records exist. If every area unexpectedly shows zero or stays on Loading, verify the browser console, API address, server process, JWT session, MariaDB service, database name, migrations, and the failing API response. Do not replace a real zero with dummy data.

10. Notifications, Notes and Follow-up

Notification events

Event	Recipient	Result
Complaint submitted	Student and administrative workflow	Confirms creation and signals new work.
Status changed	Complaint owner	Explains the new state and optional note.
Department action	Complaint owner and activity records	Keeps the student informed and records staff work.
Potential-false flag changed	Complaint owner	Discloses that a review flag or reason changed.
Account registered	New student mailbox	Sends a welcome message when SMTP is configured.
Password reset requested	Matching account mailbox	Sends a time-limited reset link without exposing account existence publicly.

Good note-writing practice

- Describe the action taken, not personal opinion about the complainant.
- State the next step and responsible department when known.
- Avoid copying unnecessary personal data into notes.
- Do not place passwords, verification codes, or secret tokens in a note.
- Keep language factual, respectful, and suitable for later audit or appeal.

11. Evidence Uploads

Supported evidence

Format	Accepted MIME type	Example use
JPEG / PNG	image/jpeg, image/png	Screenshot, notice, receipt image, or photographed document.
PDF	application/pdf	Official notice, fee document, result sheet, or written record.
MP3 / WAV	audio/mpeg, audio/wav	Voice or audio evidence where policy permits.
MP4	video/mp4	Video evidence where policy permits.

The server limit is 20 MB per attachment. Multer checks the submitted MIME type, creates a randomized server filename, stores the file on disk, and stores its original name, path, type, size, and upload time in MariaDB.

Production requirements

- Require authentication and authorization before evidence download.
- Use private persistent storage or private object storage with short-lived signed links.
- Scan untrusted files for malware and validate file content, not only the declared MIME type.
- Define retention, access, deletion, and legal-hold policies.
- Back up evidence separately from the SQL database.

Current implementation warning
The application currently serves the upload path as static content. Sensitive production evidence should not remain publicly reachable by URL. Protect or replace this delivery method before real deployment.

12. Database Design

The current logical schema contains 18 tables. The table list below describes purpose only and does not expose any stored records.

Table	Purpose
users	Student accounts, unique email and registration number, password hash, timestamps.
admins	Administrative and staff accounts with role values.
categories	Configured complaint categories.
complaints	Complaint text, owner, category, AI results, department, status, and review flag.
status_tracking	Previous and new status, actor, note, and timestamp.
departments	Operational department definitions.
department_admins	Links administrative accounts to departments.
complaint_attachments	Evidence metadata and file path.
complaint_device_records	Server-observed network and user-agent metadata for new complaints.
notifications	Messages for students or administrative accounts.
chatbot_messages	Student and assistant conversation history.
audit_logs	Important actions, actor type, description, network address, and time.
member_activity_logs	Department-member work events.
member_task_notes	Notes attached to complaint work.
email_verifications	Hashed registration code, expiry, attempts, and send time.
password_resets	Reset token, account type, expiry, and used state.
otp_codes	General or legacy one-time-code records.
universities	University reference data.

Fresh database installation order

1. Import database/schema.sql.
2. Import database/seed.sql.
3. Import database/advanced_features_migration.sql.
4. Import database/advanced_seed.sql.
5. Import database/advanced_migration.sql.
6. Import database/separate_role_dashboards_migration.sql.
7. Import database/email_verification_migration.sql.
8. Import database/normalize_registration_numbers.sql.
9. Import database/complaint_accountability_migration.sql.

Destructive schema warning
The base schema drops and recreates core tables. Use it only for a clean installation. Back up an existing database and apply only required migrations after inspecting the current structure.

13. API Overview

Local development uses the base address `http://localhost:5000/api`. Protected requests include a bearer token. The main route groups are summarized below.

Route group	Representative operations	Access
/health and /public	Health check and public overview.	Public
/auth	Verification configuration, send code, register, student login, admin login, forgot and reset password.	Public with rate limits
/complaints	Submit complaint, list own complaints, student dashboard summary.	Student
/complaints/with-attachment	Submit complaint with optional evidence.	Student
/chatbot	Send assistant message and load chat history.	Student
/admin	All complaints, detail, status, review flag, dashboard stats, analytics.	Admin roles
/advanced	Departments, notifications, department work, analytics, predictions, audit, CSV report.	Authenticated and role-scoped
/members	Work summary, activity, and complaint notes.	Staff or main admin by route
/roles	Create role accounts, list administrative accounts, department lookup.	Main admin by route
/full-admin	Delete authorized accounts or complaints.	Main admin

Common response codes

Code	Meaning
200 / 201	Successful read, update, or creation.
400	Missing, invalid, unsupported, or expired request data.
401	Token missing, invalid, expired, or login credentials rejected.
403	Authenticated role is not allowed to perform the action.
404	Route or requested record was not found.
409	A unique value such as Gmail or registration number already exists.
429	Rate limit or verification-attempt limit reached.
500-series	Application, database, SMTP, file, or external provider failure.

14. Local Installation with XAMPP

Prerequisites

- Node.js 20 or newer and npm 10 or newer.
- XAMPP with MariaDB/MySQL and phpMyAdmin, or an equivalent MariaDB service.
- A browser and a terminal opened in the project root.
- Optional Gmail SMTP credentials and optional OpenAI API credentials.

Installation steps

1. Start Apache and MySQL from XAMPP.
2. Open phpMyAdmin at http://localhost/phpmyadmin/.
3. Create the clean database by importing the SQL files in the order listed in the database section.
4. Copy .env.example to .env only when a local .env does not already exist.
5. Fill the database connection, a strong JWT secret, SMTP options, and optional AI settings.
6. Run npm install to create or update node_modules and package-lock.json.
7. Create an authorized initial administrative account using the local project procedure; do not publish its credentials.
8. Run npm run dev.
9. Open http://localhost:5000 and verify http://localhost:5000/api/health.

Useful commands

```
npm install
npm run dev
npm start
npm run check:syntax
npm audit
```

Protect local secrets
The .env file is ignored by Git and must remain private. Do not copy real database, email, JWT, or AI credentials into documentation or source control.

15. Environment and Email Configuration

Application variables

Variable group	Purpose
PORT, NODE_ENV, APP_BASE_URL	Server port, runtime mode, and public application address.
DB_HOST, DB_USER, DB_PASSWORD, DB_NAME	MariaDB connection settings.
JWT_SECRET, JWT_EXPIRES_IN	Token signature secret and token lifetime.
EMAIL_HOST, EMAIL_PORT, EMAIL_SECURE	SMTP server and transport security mode.
EMAIL_USER, EMAIL_PASS, EMAIL_FROM	SMTP identity, App Password, and sender label.
REQUIRE_EMAIL_VERIFICATION	Turns registration-code enforcement on or off.
USE_OPENAI_AI, OPENAI_API_KEY, OPENAI_MODEL	Controls optional external AI and model selection.
APP_PORT and DOCKER_DB_*	Local Docker Compose port and database values.

Gmail SMTP setup

1. Use a dedicated sending mailbox approved for the application.
2. Enable two-step verification for that mailbox.
3. Create a Gmail App Password and place it only in the private .env or hosting secret store.
4. Use SMTP port 587 with STARTTLS and EMAIL_SECURE=false, or use the provider's approved equivalent.
5. Set REQUIRE_EMAIL_VERIFICATION=true for production registration.
6. Test verification, welcome, and password-reset delivery, including the spam folder.

Verification behavior

- A valid Gmail format is required before a code request.
- A previously registered Gmail address is rejected.
- A code is valid for ten minutes and is stored as a hash.
- A new code cannot be requested continuously; a resend delay applies.
- Repeated incorrect attempts are limited and require a new code.

16. Docker and Render Deployment

Docker Compose

Docker Compose defines an application service and a MariaDB service. Named volumes preserve database data and uploaded evidence. Database health checks delay application startup until MariaDB is ready. Initialization scripts run alphabetically only when the database volume is empty.

```
docker compose up --build
docker compose exec app npm run seed:admin
docker compose down
```

Migration filename check
Before building the current Compose stack, confirm every mounted migration filename exists in the database folder. The current advanced migration file is named `advanced_migration.sql`.

Container design

- The image uses Node.js 22 Alpine and installs production packages with npm ci.
- Only runtime backend and frontend files are copied into the image.
- The process runs as the non-root node user.
- A health check calls the API health endpoint.
- The upload directory is mounted as persistent storage in Compose.

Render

The Render blueprint defines a Node web service, production build and start commands, an HTTP health check, a generated JWT secret, and names for required environment variables. It does not create a MariaDB service, so an external compatible database must be supplied.

- Set the final HTTPS site address in APP_BASE_URL.
- Enter database, email, and optional AI secrets in the Render dashboard rather than the repository.
- Use a persistent disk or private object storage for evidence.
- Apply the database schema and migrations before expecting live dashboards to work.
- Change the browser API base from localhost to the deployed API origin or relative /api path.

17. Security, Privacy and Accountability

Current security controls

- bcrypt password hashing.
- Signed JWT authentication and backend role middleware.
- Helmet browser security headers.
- Production CORS restricted to APP_BASE_URL.
- Authentication and verification rate limits.
- Unique Gmail and registration number constraints.
- Hashed verification codes and expiring reset tokens.
- MIME filtering and file-size limits for uploads.
- Audit, status, notification, and member-activity records.

Complaint device records

For new complaints, the server records the complaint and student identifiers, observed network IP address, browser user-agent string, parsed device type, browser name, operating system, and recording time. This supports investigation and abuse review. It does not collect a guaranteed hardware identifier or MAC address.

Metadata limitation
Network addresses and user-agent details can be shared, proxied, changed, or spoofed. Device metadata and AI fake probability are supporting signals only. They cannot independently establish identity, intent, or wrongdoing.

Production privacy requirements

- Publish a clear privacy notice covering purpose, access, retention, appeals, correction, and deletion.
- Use HTTPS and a strong random JWT secret.
- Use a least-privilege database account and restrict phpMyAdmin access.
- Protect evidence behind authorization or signed private links.
- Do not log passwords, tokens, verification codes, or unnecessary complaint content.
- Review access to exports, audit logs, account lists, and device metadata.
- Require human review and an opportunity to respond before adverse action.

18. Operations, Backup and Maintenance

Daily operations

- Confirm the API health endpoint and public landing page are available.
- Review pending and high-priority queues without assuming AI labels are correct.
- Check failed email delivery and rate-limit events.
- Ensure department members add clear notes and students receive follow-up notifications.
- Monitor disk space for evidence uploads and database capacity.

Backup plan

Asset	Backup method	Restore test
MariaDB	Scheduled SQL dump or managed database backup.	Restore to a separate test database and verify tables and counts.
Evidence files	Filesystem or object-storage snapshot.	Verify database paths resolve to restored files.
Environment secrets	Approved secret manager or encrypted operational record.	Confirm deployment can be rebuilt without exposing secret values.
Source code	Private version-control repository.	Build from a clean checkout using package-lock.json.

Dependency maintenance

```
npm outdated
npm audit
npm prune
npm dedupe
npm ci --dry-run --ignore-scripts
```

Review major-version migration notes before updating. After an update, test authentication, bcrypt compatibility, uploads, rate limiting, security headers, database-backed routes, email transport, and static pages.

19. Troubleshooting

Symptom	Likely causes	Recommended checks
Site stays on Loading	JavaScript error, wrong API address, server unavailable, or failing endpoint.	Open browser console and Network tab; test /api/health; verify API base and server log.
Failed to fetch	Node server stopped, wrong port, CORS/origin issue, or localhost used from another device.	Start the server; verify port; use the correct host address; check production APP_BASE_URL.
Student cannot register	Invalid Gmail, duplicate Gmail or registration number, password validation, missing verification, or database failure.	Read the API response; verify SMTP and migrations; do not repeatedly request codes.
Verification email not sent	SMTP values missing, normal password used instead of App Password, provider rejection, or rate limit.	Check private SMTP configuration and server log; wait for limiter; inspect spam folder.
Too many verification requests	Repeated requests within the configured limit.	Wait for the limit window; avoid repeated button clicks; restart only for local testing.
All dashboard values are zero	No matching records, wrong account, wrong database, or API query failure.	Confirm records in the correct database and inspect the endpoint response.
EADDRINUSE on port 5000	Another process already owns the port.	Use the existing server, stop the correct process, or change PORT.
rs not recognized	Command was entered at a normal PowerShell prompt.	Use rs only inside the active nodemon session; otherwise run npm run dev.
Docker database is empty	Named volume already initialized before migration changes.	Back up data; inspect the volume; reinitialize only disposable local volumes.
Evidence disappears after deployment	Ephemeral filesystem or missing persistent volume.	Use persistent disk or private object storage and verify backups.

Safe diagnostic order

1. Read the exact browser or server error.
2. Check /api/health.
3. Verify the application port and API base address.
4. Confirm MariaDB is running and the configured database exists.
5. Inspect the failing endpoint response and server log.
6. Check token role and session storage for protected routes.
7. Verify required migrations and file permissions.
8. Retest with a minimal, privacy-safe input.

20. Release and Acceptance Checklist

Functional acceptance

- First-visit introduction and public landing load correctly.
- Student registration enforces unique Gmail and registration number rules.
- Verification, welcome, and password-reset email work when SMTP is enabled.
- Student login accepts Gmail and registration number.
- Student session persistence and administrative session behavior match policy.
- Complaint submission stores analysis, status, notification, and new request metadata.
- Evidence validation rejects unsupported formats and files above the limit.
- Student dashboards, all complaints, status, and notifications show live data.
- Authorized status changes and notes notify the complaint owner.
- Role dashboards cannot access work outside their permitted scope.
- Analytics, predictions, audit, member activity, and CSV export enforce role permissions.

Security and privacy acceptance

- No default or shared credentials remain in production.
- No .env file or secret is committed to source control.
- HTTPS, CORS, JWT secret, database permissions, and rate limits are configured.
- Evidence is private, persistent, scanned, and recoverable.
- Privacy notices and retention rules cover complaints, chats, attachments, logs, and device metadata.
- AI and metadata outputs are labeled advisory and reviewed by a human.
- Backups have been restored successfully in a test environment.

Final operating principle

A complaint system is a trust system
Technical features are only part of the solution. Fair access, timely follow-up, respectful communication, data minimization, human judgment, transparent reasons, and a meaningful appeal path determine whether the system earns trust.

Appendix A. Main Page Map

Area	Path	Purpose
Public	/	Landing page and public overview.
Introduction	/welcome.html	First-visit explanation.
Student login	/user/login.html	Student authentication.
Student registration	/user/register.html	New student account and verification.
Student dashboard	/user/dashboard.html	Live personal summary.
Submit complaint	/user/submit-complaint.html	Complaint and evidence submission.
All complaints	/user/complaint-history.html	Complete student complaint history.
Status	/user/status.html	Complaint progress view.
Notifications	/user/notifications.html	Follow-up messages.
Assistant	/user/chatbot.html	Contextual complaint guidance.
Admin login	/admin/admin-login.html	Separate administrative and staff authentication.
Admin dashboard	/admin/dashboard.html	System overview and operations.
Complaint management	/admin/complaints.html	Review, filters, detail, status, and flags.
Analytics	/admin/analytics.html	Live reports and trends.
Advanced analytics	/admin/advanced-analytics.html	Extended distributions and insight.
Predictions	/admin/predictions.html	AI-supported current summaries.
Role management	/admin/roles.html	Authorized role-account management.
Member activity	/admin/member-activity.html	Department-member work reporting.
Department dashboard	/department/dashboard.html	Department Admin work.
Faculty dashboard	/faculty/dashboard.html	Faculty Staff work.
Reviewer dashboard	/reviewer/dashboard.html	Reviewer work.

Appendix B. Glossary

Term	Meaning
API	The HTTP interface used by browser pages to request or change system data.
Bearer token	A signed JWT sent in the Authorization header for protected API calls.
CORS	Browser rule controlling which web origins can call the API.
Dashboard	A role-specific page presenting current counts, records, or work.
Department scope	The set of complaints a department role is allowed to access.
Evidence	An optional uploaded image, document, audio file, or video supporting a complaint.
Fallback	The local result used when optional external AI is disabled or fails.
Human review	A responsible person's examination before a decision or action.
JWT	A signed token carrying account identifier and role for API authentication.
Live data	Current API and database results rather than fixed demonstration content.
MariaDB	The relational database engine verified for the current local installation.
Migration	SQL that adds or changes database structures for a feature.
MIME type	The submitted content type used as one input to upload validation.
NLP	Text-analysis logic used for category, sentiment, and priority estimation.
Rate limit	A restriction on repeated requests within a time window.
SMTP	The mail transport used for verification, welcome, and reset messages.

Appendix C. Privacy-Safe Documentation Rules

- Use generic role names instead of real account identities.
- Use empty or clearly fictional placeholders instead of real credentials.
- Do not publish database screenshots containing names, contact details, complaint text, or metadata.
- Redact attachments, registration numbers, network addresses, and audit descriptions before sharing evidence of a bug.
- Share the minimum log lines needed to diagnose an error and remove tokens or secrets.
- Store operational documentation according to university access and retention policy.

End of guide. This document describes the current application structure and should be reviewed whenever authentication, roles, AI behavior, storage, database migrations, or deployment architecture changes.